Introduction

Edge detection is a fundamental technique in image processing and computer vision. It helps identify the boundaries or edges within an image where there is a significant change in pixel intensity. This exercise focuses on gradient-based operators which compute the intensity gradients and highlight the regions of abrupt intensity changes.

Gradient Operators

Gradient-based methods detect edges by looking for significant changes in the intensity gradient of the image.

Common operators:

- Sobel Operator: Uses separate kernels in the x and y direction. It is often used for detecting vertical and horizontal edges distinctly.
- Prewitt Operator: Conceptually similar to Sobel but uses simpler kernels.
- Roberts Operator: An older operator using diagonal gradients. Often more sensitive to noise.
- Scharr Operator: A variant of Sobel providing better rotational symmety

Key Parameters and Concepts

- Kernel Size: Determines the neighborhood considered in calculating the gradient. Larger kernels smooth more but may lose some small or fine edges.
- Thresholding: Post-processing step to decide whether a gradient magnitude is strong enough to be considered an edge. A single threshold or a double threshold approach can be used.
- Noise Sensitivity: Operators can pick up edges caused by noise. Pre-smoothing (e.g., Gaussian blur) is often used to reduce noise before edge detection.
- Directionality: Sobel is direction-aware (calculates derivative in x and y directions). Analyzing these allows you to see horizontal and vertical edges separately or combine them to get a total gradient magnitude.

Key Observations:

• Larger kernels can blur small edges.

• Lower threshold includes more edges, potentially adding noise.

• Higher threshold excludes weak edges, but may lose detail in low-contrast areas

EXCERCISES

5.1



Figure1: Original image



Figure2: With Gaussian blur kernel size 21

5.2

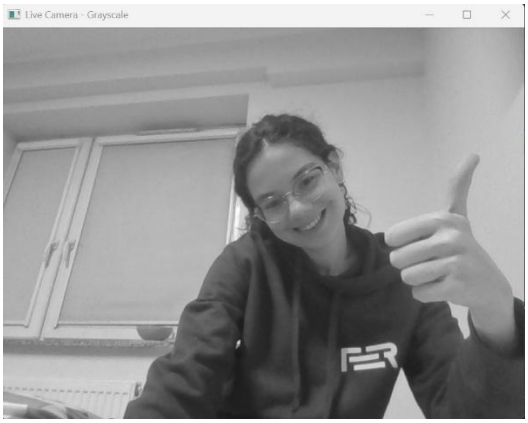
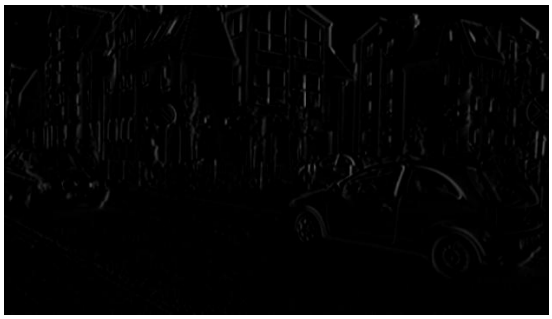


Figure3: Screenshot of live webcam using OpenCV

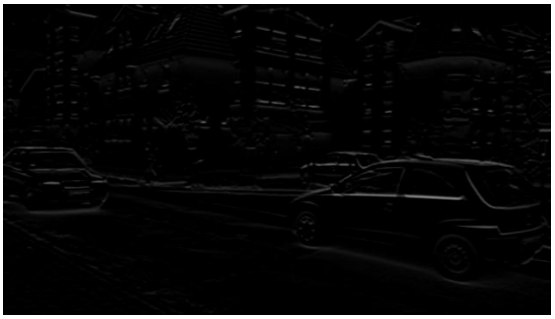
5.3

Images shown below are all done using a Gaussian blur with kernel size 21 and Sobel edge detection with ksize = 3 without tresholding.

PREWITT



Prewitt_x



Prewitt_y



Prewitt_combined

SOBEL



Sobel_x



Sobel_y



Sobel_combined

SCHARR





Scharr_x Scharr_y



Scharr_combined

5.4 Parameter Analysis given in part 6

5.5 Live Testing on Camera Feed



It depends what we want and what we consider as an edge. In my opinion, both of these pictures are good depending on what we want to emphasize (only a sulluete or more details) The first one is created with sobel kernel 5, treshold 75 and moderate blur and the other one with sobel kernel size 3, treshold 12 and blur 1.

6. Discussion and Concluding Question

Sobel operator performed better than Prewitt but the best one was Scharr. Sobel combined was the best but, if we want to compare X and Y, it is shown that Y is better for the original image because it has more horizontal structures (building edges and road tracks). This results in stronger vertical intensity gradients. The same logic applies for other operators too. Scharr operator performed better then Prewitt and Sobel.

In my experiments I have concluded:

Low threshold detects many edges, includes noise and unnecessary details and image looks cluttered or granular. The image is almost all white with treshold zero.

Medium threshold is the best balance between noise and details

With high threshold (above 100) only strongest edges remain and too much detail is lost.

In terms of implementation:

- **Prewitt** is simpler but slightly less accurate and more sensitive to noise
- **Sobel** is widely used and balances detail and noise well
- **Scharr** improves gradient calculation for small features, giving the sharpest edges with minimal noise

Regarding kernel size I got the best results with the size 5. With small sizes it is detailed but noisy and with larger the result is cleaner but less detailed. I also tried with different Gaussian kernel size and compared how it performed.

For the chosen image, Scharr combined with Sobel-like weighting and a moderate kernel size 3 with medium Gaussian blur produced the clearest and most balanced edges.

But also with blur 3 (instead of 21 like in the beginning) and kernel 5 I got a very detailed image that I think looks most like the original. But it depends if that is what we wanted or just to detect edges. The image is shown below:

